information in a later layer. We did not do that here for simplicity, but is a good exercise for you to try. In listing 9.27 we define a function to choose the action for an agent, given its observation (the mean field of its neighboring agents).

Listing 9.17. Choosing actions

```
def
infer_acts(obs,param,layers,pos_list,acts,act_space=21,num_iter=5,t
emp=0.5):
    N = acts.shape[0]
1
    mean_fields = torch.zeros(N,act_space)
    acts_ = acts.clone()
2
    qvals = torch.zeros(N,act_space)

    for i in range(num_iter):
3
        for j in range(N):
4
            mean_fields[j] = get_mean_field(j,pos_list,acts_)

        for j in range(N):
5
            state = torch.cat((obs[j].flatten(),mean_fields[j]))
            qs = qfunc(state.detach(),param,layers=layers)
            qvals[j,:] = qs[:]
            acts_[j] = softmax_policy(qs.detach(),temp=temp)
    return acts_, mean_fields, qvals

def init_mean_field(N,act_space=21):
6
    mean_fields = torch.abs(torch.rand(N,act_space))
    for i in range(mean_fields.shape[0]):
        mean_fields[i] = mean_fields[i] / mean_fields[i].sum()
    return mean_fields
```

- ***1*** Gets the number of agents
- ***2*** Clones the action vector to avoid changing in place
- ***3*** Alternates a few times to converge on action
- ***4*** Loops through the agents and computes their neighborhood mean field action vectors
- ***5*** Uses the mean field actions and state to compute Q values and select actions using a softmax policy
- ***6*** Randomly initializes the mean field vectors